UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

POSGRADOS

Maestría en

SOFTWARE

Asignatura:

Patrones de Diseño de Software

Actividad #: 02

Patrones estructurales

Mauricio Ortiz Ochoa

Material para uso académico exclusivo de Mauricio Ortiz Ochoa, mortizo@ups.edu.ec

Actividad de aprendizaje #: 02

Patrones estructurales

Fecha

- 2026/10/22

Grupo base

- Grupo 06

Integrantes

- Cabrera Bonilla Brando Daniel
- Cabrera Carbajal Wilson Alexander
- Espinoza López Cristian Paúl
- Jiménez Quevedo Cristian Geovanny

Resultado de aprendizaje

- RA2. Modela soluciones de software aplicando patrones estructurales de acuerdo con el contexto y las necesidades de diseño.

Entregables:

- Archivo en PDF con el siguiente nombre: PDS_Actividad_02_GrupoXX.pdf
- Aplicación de Software almacenado en repositorio de código.
- La actividad se desarrolla en grupo; la carga al AVAC es individual.

Tareas específicas:

- Participar en la clase de la Unidad 02, secciones 01, 02 y 03.

- Leer detenidamente el material de apoyo relacionado.
- Crear un repositorio de código para almacenar la aplicación solicitada.
- Seguir las instrucciones del ejercicio.

Instrucciones:

Escenario 01:

Una institución necesita representar la estructura de su presupuesto mediante diferentes niveles jerárquicos.

Existen partidas presupuestarias individuales, que contienen información como código, descripción y valor asignado. Estas partidas representan los elementos básicos sobre los cuales se registran los recursos disponibles.

Sin embargo, para organizar el presupuesto también existen grupos presupuestarios. Un grupo puede contener varias partidas individuales, pero también puede contener otros grupos presupuestarios, formando una estructura jerárquica de varios niveles.

Por ejemplo, un grupo denominado Tecnologías de la Información podría contener partidas relacionadas con licenciamiento, infraestructura y servicios. A su vez, el grupo Infraestructura podría contener otros grupos o partidas correspondientes a servidores, almacenamiento y comunicaciones.

Tanto las partidas individuales como los grupos deben permitir realizar operaciones comunes, entre ellas:

- obtener su código y descripción;
- calcular su valor total;
- mostrar su estructura.

En una partida individual, el valor total corresponde directamente al valor asignado. En un grupo presupuestario, el valor total corresponde a la suma de los valores de todos los elementos que contiene, incluyendo aquellos que formen parte de grupos internos.

Actualmente, el código cliente tendría que identificar constantemente si está trabajando con una partida individual o con un grupo para decidir cómo calcular su valor o recorrer sus componentes. Se desea evitar esta distinción y permitir que ambos tipos de elementos puedan tratarse mediante una misma abstracción.

Además, la estructura debe poder crecer incorporando nuevos niveles de agrupación sin modificar la lógica principal que consulta o procesa el presupuesto.

Diseñar una solución orientada a objetos que permita representar la estructura jerárquica, realizar operaciones comunes sobre elementos individuales y agrupaciones y calcular recursivamente los valores correspondientes.

Escenario 02:

Una Universidad dispone de un sistema institucional para registrar y consultar la asistencia de sus colaboradores.

El sistema interno utiliza un contrato común mediante el cual solicita los registros de una persona para una fecha determinada. Como resultado espera obtener información normalizada, por ejemplo:

- identificación del colaborador;
- fecha;
- hora de ingreso;
- hora de salida.

Durante varios años se han utilizado dispositivos biométricos de un proveedor que ya trabaja de acuerdo con esta estructura.

Sin embargo, la universidad adquiere dispositivos de otro fabricante. Estos equipos incluyen una biblioteca propia que no puede modificarse y cuya interfaz es diferente. Por ejemplo, en lugar de consultar directamente la asistencia diaria, la biblioteca permite recuperar una colección de marcaciones mediante operaciones y tipos de datos propios del fabricante.

Los nombres de las operaciones, los parámetros requeridos y las estructuras devueltas no coinciden con la interfaz que utiliza el sistema institucional.

Modificar todo el sistema para trabajar directamente con la nueva biblioteca produciría dependencia respecto del proveedor. Tampoco resulta conveniente modificar el software entregado por el fabricante, porque podría perderse compatibilidad con futuras versiones.

Además, en el futuro podrían adquirirse equipos de otros proveedores con interfaces nuevamente diferentes.

La universidad necesita mantener estable el contrato utilizado por su sistema y permitir que los nuevos dispositivos puedan incorporarse sin que el resto de la aplicación conozca sus particularidades.

Diseñar una solución orientada a objetos que permita utilizar el nuevo dispositivo mediante la interfaz que ya espera el sistema institucional, realizando las transformaciones necesarias entre ambos componentes.

Para ambos escenarios, cada grupo deberá:

- Identificar el problema estructural que presenta el escenario.
- Seleccionar y justificar el patrón estructural correspondiente.
- Elaborar el diagrama de clases UML.
- Identificar los participantes del patrón dentro de la solución propuesta.
- Implementar la solución en un lenguaje de programación.
- Mantener correspondencia entre el modelo UML y la implementación.
- Almacenar el código fuente en el repositorio del grupo.

Para cada escenario, documentar el uso crítico de IA mediante:

- Prompt principal.
- Una recomendación aceptada.
- Una recomendación modificada o rechazada.
- Justificación de ambas decisiones.

Escenario 01

Justificación

Problema estructural identificado
La institución debe representar su presupuesto como una **jerarquía parte-todo** de profundidad variable:

- Las **partidas presupuestarias** son los elementos básicos (código, descripción y valor asignado).
- Los **grupos presupuestarios** contienen partidas y también otros grupos, formando varios niveles (por ejemplo, Tecnologías de la Información -> Infraestructura -> Servidores).

Ambos tipos deben responder a las mismas operaciones: obtener código y descripción, calcular su valor total y mostrar su estructura. Sin un diseño adecuado, el código cliente tendría que preguntar en cada paso "¿esto es una partida o un grupo?" para decidir cómo calcular el valor o cómo recorrer sus componentes.

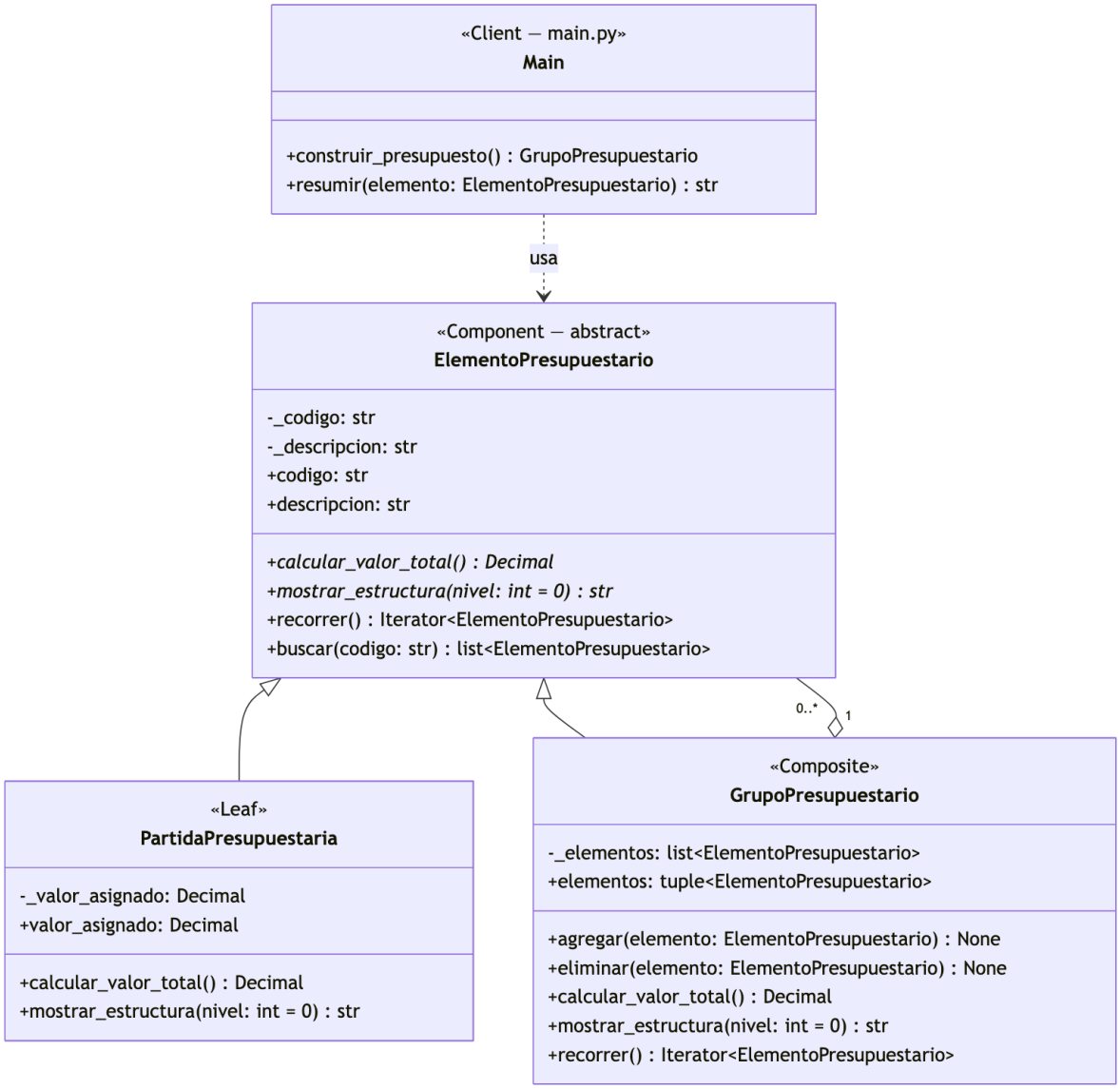
Esa distinción:
- Se repite en cada operación (condicionales if/isinstance dispersos);
- Obliga a modificar el cliente cada vez que aparece un nuevo nivel de agrupación, violando el principio Abierto/Cerrado;
- No escala a jerarquías de profundidad arbitraria sin escribir recorridos recursivos en el propio cliente.

Patrón seleccionado: Composite

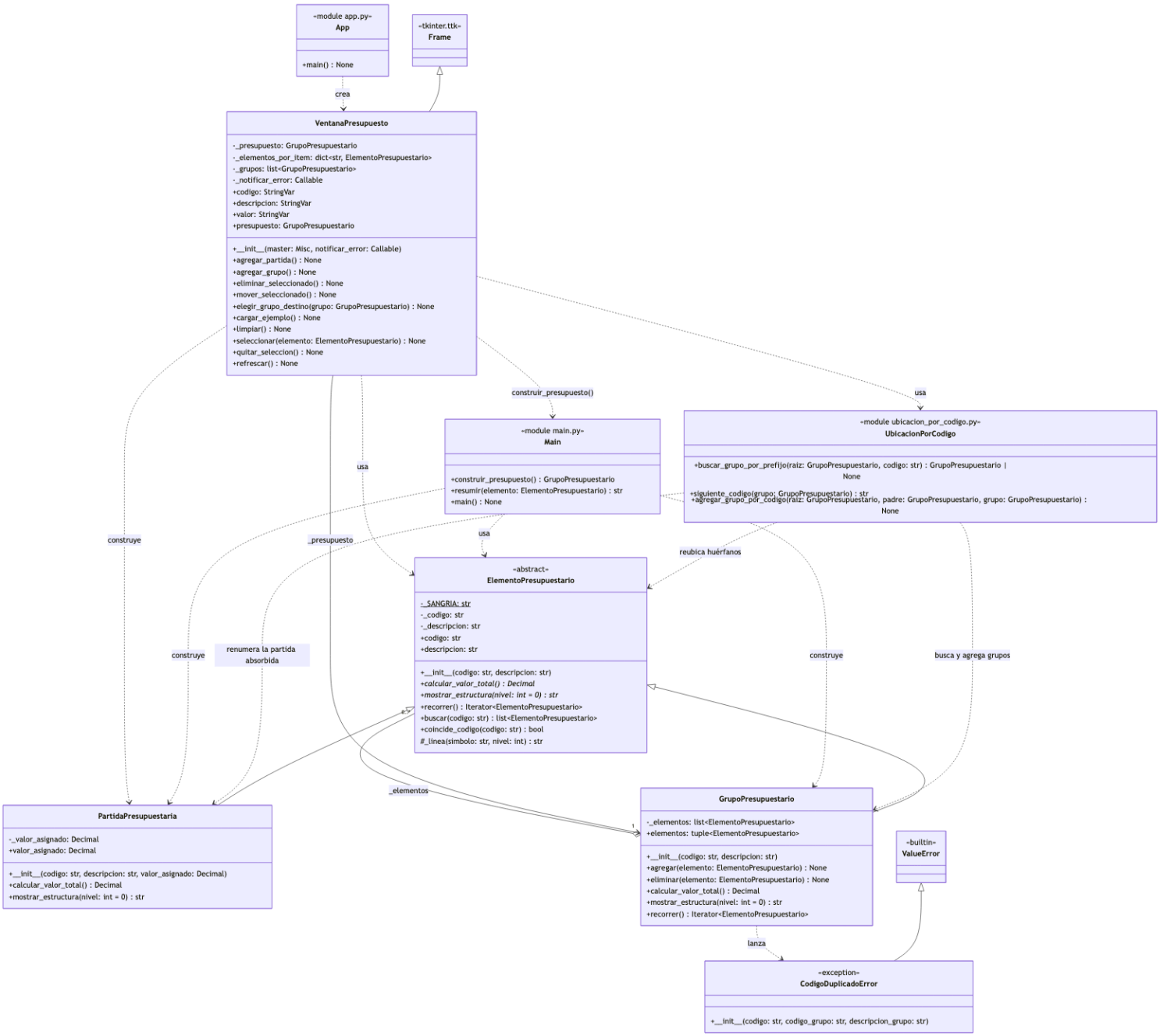
El patrón estructural Composite tiene como intención componer objetos en estructuras de árbol para representar jerarquías parte-todo y permitir que los clientes traten de manera uniforme a los objetos individuales y a sus composiciones. Coincide punto por punto con lo que pide el escenario:

Requisito del escenario	Cómo lo resuelve Composite
Partidas y grupos tratados con una misma abstracción	Ambos heredan de **ElementoPresupuestario** (Component); el cliente solo conoce esa abstracción.
Un grupo contiene partidas y otros grupos	**GrupoPresupuestario** agrega una lista de **ElementoPresupuestario**, no de partidas: puede contener cualquier elemento.
Valor total de una partida = valor asignado	**PartidaPresupuestaria.calcular_valor_total()** devuelve **valor_asignado**.
Valor total de un grupo = suma recursiva de sus elementos	**GrupoPresupuestario.calcular_valor_total()** delega en cada hijo y suma; la recursión vive en el Composite, no en el cliente.
Mostrar la estructura	**mostrar_estructura(nivel)** se implementa en hoja y compuesto; el grupo se muestra a sí mismo y delega en sus hijos con un nivel más de sangría.
Crecer sin modificar la lógica principal	Agregar niveles es solo agregar grupos dentro de grupos. Prueba de ello: se incorporó una interfaz gráfica completa como nuevo cliente **sin modificar ninguna clase del patrón**.

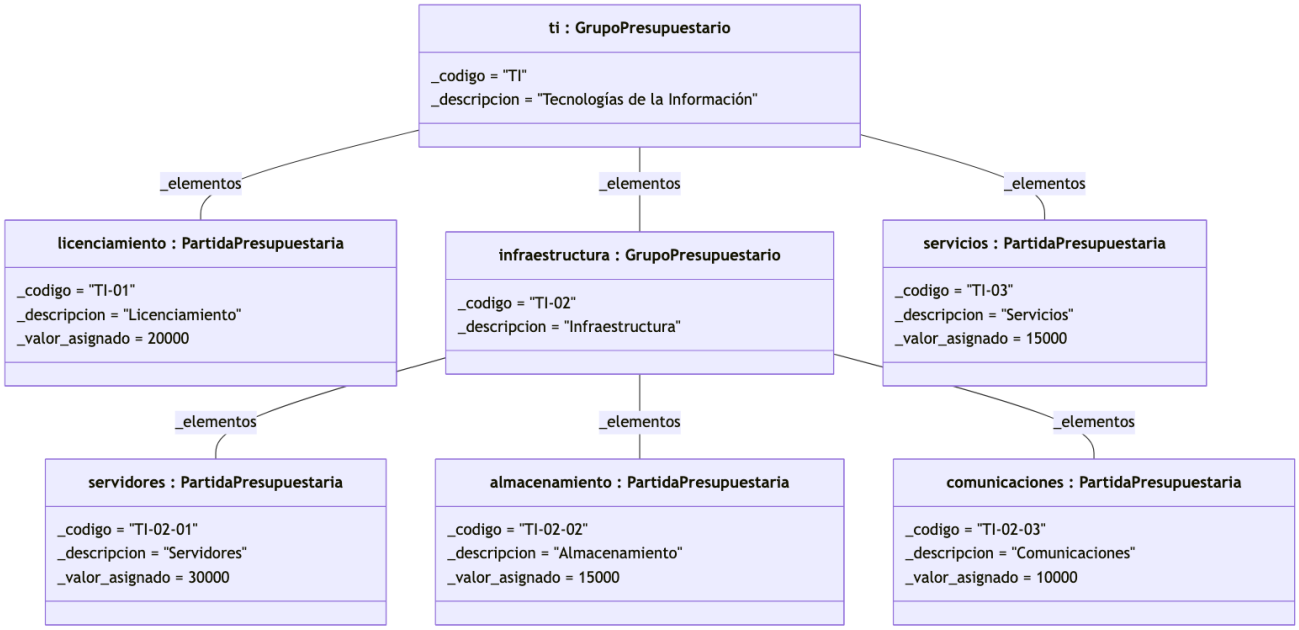
- **Participantes del patrón Composite**



- **Vista completa de la solución** (incluye clientes, reglas de codificación y excepción de dominio).



- **Diagrama de objetos del ejemplo del enunciado** (instancias creadas por construir_presupuesto()).



Correspondencia UML – código

Elemento UML	Implementación
Clase abstracta **ElementoPresupuestario**	**class ElementoPresupuestario(ABC)** en **presupuesto/elemento_presupuestario.py**
Generalización Leaf / Composite	**class PartidaPresupuestaria(ElementoPresupuestario)** y **class GrupoPresupuestario(ElementoPresupuestario)**
Agregación **_elementos 0..***	**self._elementos: list[ElementoPresupuestario]** en **GrupoPresupuestario**
Operaciones abstractas (*cursiva*)	**@abstractmethod calcular_valor_total()** y **mostrar_estructura()**
Objetos del diagrama de objetos	Variables **ti, licenciamiento, infraestructura, servidores, almacenamiento, comunicaciones, servicios** en **main.construir_presupuesto()**

Código fuente

- **Repositorio:** https://github.com/NightMare2210/PDS_Actividad_02 (carpeta Escenario_1_Composite/)
- **Lenguaje:** Python 3 (solo biblioteca estándar; la interfaz gráfica usa tkinter)

```
Escenario_1_Composite/
├── presupuesto/                    # Dominio
│   ├── elemento_presupuestario.py  # Component
│   ├── partida_presupuestaria.py   # Leaf
│   ├── grupo_presupuestario.py     # Composite
│   ├── codigo_duplicado_error.py   # Excepción de dominio
│   └── ubicacion_por_codigo.py     # Reglas de codificación
├── interfaz/ventana_presupuesto.py # Client gráfico
├── main.py                         # Client por consola
├── app.py                          # Lanzador de la interfaz gráfica
├── tests/                          # 73 pruebas unitarias
└── diagrama.md                     # Diagramas UML (Mermaid)
```

```python
class ElementoPresupuestario(ABC):
    """Component: abstracción común para partidas y grupos presupuestarios."""

    def __init__(self, codigo: str, descripcion: str) -> None:
        if not codigo.strip():
            raise ValueError("El código es obligatorio")
        self._codigo = codigo
        self._descripcion = descripcion

    @property
    def codigo(self) -> str:
        return self._codigo

    @property
    def descripcion(self) -> str:
        return self._descripcion

    @abstractmethod
    def calcular_valor_total(self) -> Decimal: ...

    @abstractmethod
    def mostrar_estructura(self, nivel: int = 0) -> str: ...


class PartidaPresupuestaria(ElementoPresupuestario):
    """Leaf: elemento básico del presupuesto con un valor asignado."""

    def calcular_valor_total(self) -> Decimal:
        return self._valor_asignado

    def mostrar_estructura(self, nivel: int = 0) -> str:
        return self._linea("-", nivel)


class GrupoPresupuestario(ElementoPresupuestario):
    """Composite: agrupa partidas u otros grupos y delega en ellos sus operaciones."""

    def agregar(self, elemento: ElementoPresupuestario) -> None:
        if any(descendiente is self for descendiente in elemento.recorrer()):
            raise ValueError("Un grupo no puede contenerse a sí mismo ni a uno de sus ancestros")
        if any(hijo.coincide_codigo(elemento.codigo) for hijo in self._elementos):
            raise CodigoDuplicadoError(elemento.codigo.strip(), self.codigo, self.descripcion)
        self._elementos.append(elemento)

    def calcular_valor_total(self) -> Decimal:
        return sum((elemento.calcular_valor_total() for elemento in self._elementos), Decimal("0"))

    def mostrar_estructura(self, nivel: int = 0) -> str:
        hijos = (elemento.mostrar_estructura(nivel + 1) for elemento in self._elementos)
        return "\n".join([self._linea("+", nivel), *hijos])
```

Ejecución:

```
cd Escenario_1_Composite
python app.py                               # interfaz gráfica
python main.py                              # demostración por consola
python -m unittest discover -s tests -t .   # pruebas (73 OK)
```

Salida de main.py:

```
=== Estructura del presupuesto ===
+ [TI] Tecnologías de la Información: 90,000.00
  - [TI-01] Licenciamiento: 20,000.00
  + [TI-02] Infraestructura: 55,000.00
    - [TI-02-01] Servidores: 30,000.00
    - [TI-02-02] Almacenamiento: 15,000.00
    - [TI-02-03] Comunicaciones: 10,000.00
  - [TI-03] Servicios: 15,000.00

=== Consulta uniforme ===
TI | Tecnologías de la Información | total: 90,000.00
TI-01 | Licenciamiento | total: 20,000.00
TI-02 | Infraestructura | total: 55,000.00
TI-03 | Servicios | total: 15,000.00
```

Uso crítico de inteligencia artificial

Herramienta utilizada: Claude Code (asistente de programación). La IA se usó como apoyo para diseñar, implementar y probar; las decisiones finales y la validación del comportamiento quedaron a cargo del grupo.

- **Prompt principal:**

"Analiza el Escenario 01 del archivo .md proporcionado, identifica el problema estructural y propón una solución aplicando el patrón de diseño Composite, representada mediante un diagrama UML e implementada en Python".

- **Recomendación aceptada:**

Implementar la variante "segura" de Composite: declarar agregar() y eliminar() solo en GrupoPresupuestario y no en la abstracción común ElementoPresupuestario.

Justificación de la aceptación: en la variante "transparente", PartidaPresupuestaria heredaría un método agregar() que no tiene sentido para ella y que solo podría lanzar una excepción en tiempo de ejecución. Con la variante segura, el modelo refleja el dominio (una partida no contiene elementos) y el error se evita por diseño. Se verificó que el costo de esta decisión es aceptable: solo quien construye el árbol necesita conocer GrupoPresupuestario, mientras que las consultas que exige el escenario (código, descripción, valor total y estructura) siguen siendo totalmente uniformes a través de ElementoPresupuestario.

- **Recomendación modificada o rechazada:**

Recomendación de la IA: al agregar un elemento en la interfaz gráfica, la selección del árbol debía tener prioridad sobre el prefijo del código; la ubicación por prefijo (PR-01 → grupo PR) solo se aplicaba si no había nada seleccionado.

Decisión: Rechazada y reemplazada, al probar la aplicación, el grupo detectó un comportamiento incorrecto: después de crear el grupo PR la interfaz dejaba seleccionada la raíz, y la partida PR-01 terminaba fuera de PR. Se reprodujo el caso y se invirtió la regla: si el prefijo del código existe como grupo, ese grupo tiene prioridad; la selección solo se usa cuando el prefijo no existe. Además se agregó que, al crear un grupo, se reubiquen automáticamente los elementos existentes con su prefijo y que una partida con el mismo código que el grupo pase a ser su primera subpartida numerada (PR -> PR-01).

Justificación: la recomendación original era técnicamente coherente, pero producía un resultado contrario a la convención de codificación de la institución, que es la que el usuario espera. La IA no tenía forma de conocer esa expectativa: el error solo se descubrió probando la aplicación con casos reales, lo que confirma que las propuestas de la IA deben validarse y no aceptarse de forma automática.

Escenario 02

Justificación

Problema estructural identificado
El sistema institucional consulta la asistencia mediante un **contrato estable**: solicita los registros de un colaborador para una fecha y espera un resultado normalizado (identificación, fecha, hora de ingreso y hora de salida). Los dispositivos del proveedor anterior ya cumplen ese contrato.

La biblioteca del nuevo fabricante resuelve la misma necesidad, pero con una **interfaz incompatible** que la universidad no puede modificar:

Aspecto	Contrato institucional	Biblioteca del fabricante
Operación	obtenerRegistro(idColaborador, fecha)	getMarcaciones(empleadoId, desde, hasta)
Identificación	String ("100")	long (100L)
Fecha	**LocalDate** (un día)	rango Date desde – Date hasta (java.util.Date)
Resultado	Optional<RegistroAsistencia> con ingreso y salida	List<Marcacion> con timestamp y tipo ("ENTRADA" / "SALIDA")

Las alternativas directas son inadecuadas: adaptar todo el sistema a la biblioteca crea **dependencia del proveedor** y obliga a repetir el cambio con cada nuevo fabricante; modificar la biblioteca **no es posible** y rompería la compatibilidad con sus versiones futuras. El problema es una **incompatibilidad de interfaces** entre un cliente que no debe cambiar y un componente que no se puede cambiar.

Patrón seleccionado: Adapter

El patrón estructural Adapter tiene como intención convertir la interfaz de una clase en otra interfaz que los clientes esperan, permitiendo que trabajen juntas clases que de otro modo no podrían por tener interfaces incompatibles. Responde a cada requisito del escenario:

Requisito del escenario	Cómo lo resuelve Adapter
Mantener estable el contrato del sistema	**AsistenciaTarget** no cambia; el cliente sigue llamando a **obtenerRegistro().**
Usar el nuevo dispositivo sin modificar su biblioteca	**AdaptadorAsistencia** recibe una **BibliotecaBiometrica** y solo usa su API pública.
Realizar las transformaciones necesarias	El adaptador convierte el ID, calcula el rango del día, invoca **getMarcaciones()** y reduce la lista de marcaciones a un **RegistroAsistencia.**
El resto de la aplicación no conoce al fabricante	**ConsolaUI** solo depende de **AsistenciaTarget**; el único tipo del fabricante que importa el adaptador es **BibliotecaBiometrica** (y **Marcacion**).
Incorporar otros proveedores en el futuro	Cada nuevo fabricante requiere únicamente un nuevo adaptador que implemente **AsistenciaTarget**; **SistemaAsistencia** no cambia (lo demuestra **SistemaAsistenciaTest.tercerProveedor**).

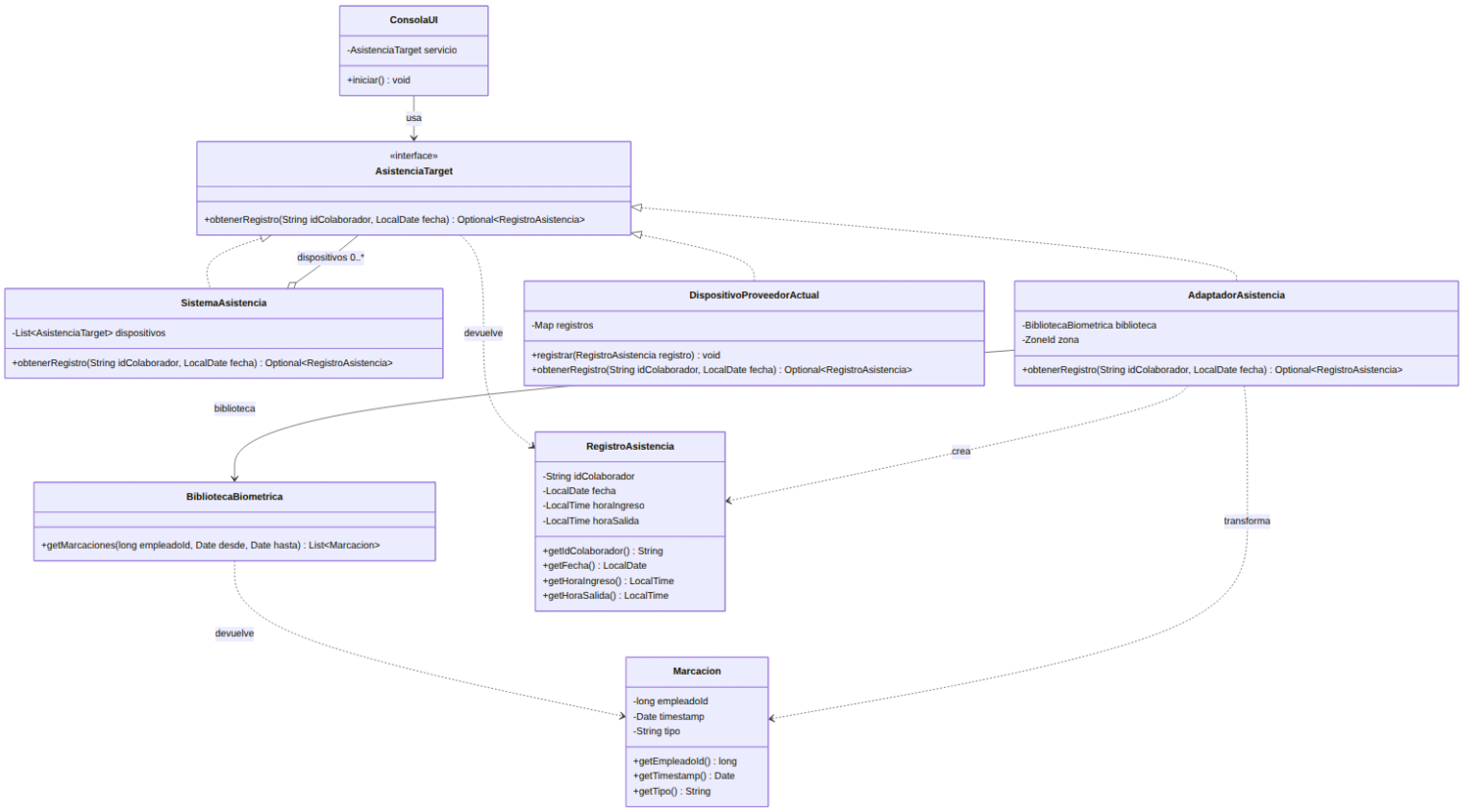
Variante elegida: Adapter de objetos

Se implementó el **Adapter de objetos** ((composición) y no el de clases (herencia): **AdaptadorAsistencia** contiene una instancia de **BibliotecaBiometrica**. Así:

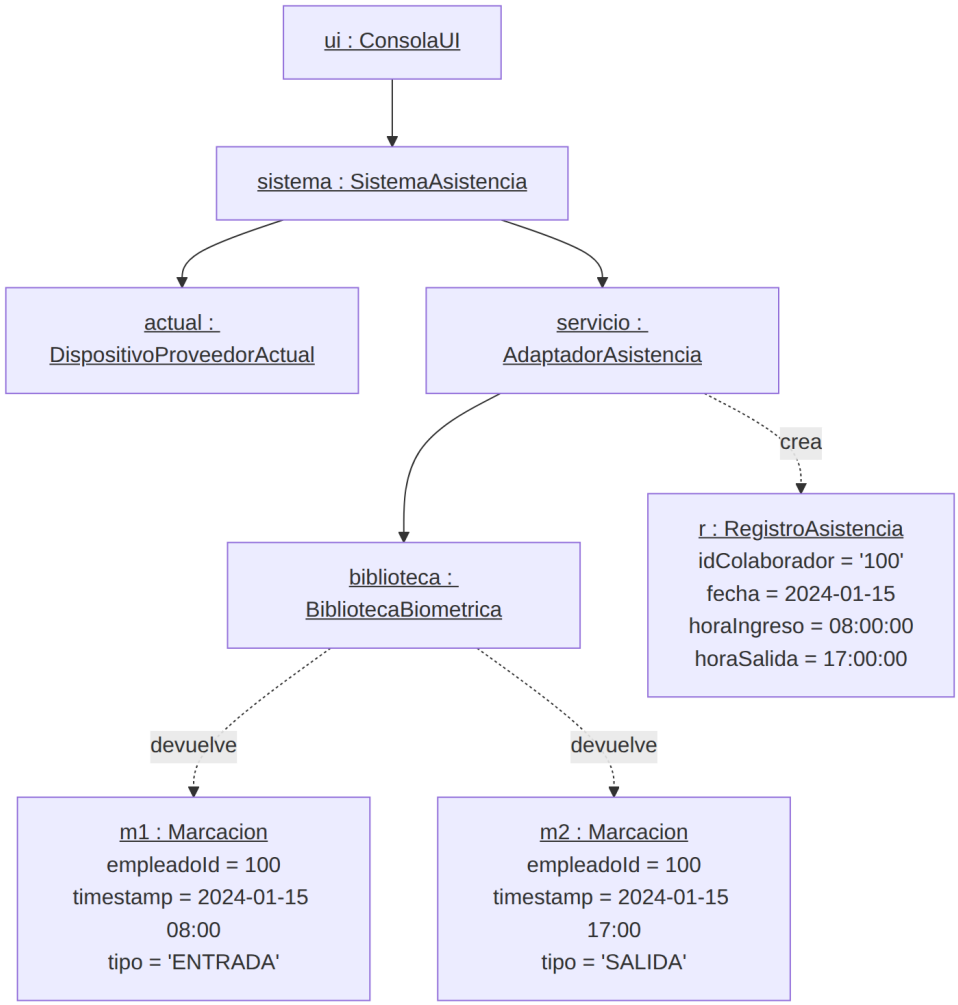
- No hereda ni expone al sistema los métodos del fabricante;
- puede envolver cualquier instancia ya configurada de la biblioteca (por ejemplo, un equipo concreto o un doble de prueba);
- No queda acoplado a la jerarquía interna de la biblioteca, que puede cambiar entre versiones. Además, Java no permite heredar de dos clases, por lo que la variante de clases solo sería posible si el Target fuera una interfaz, y aun así acoplaría el adaptador a la implementación del fabricante.

Diagrama de clases

- **Diagrama de clases del patrón Adapter**



- **Diagrama de objetos del ejemplo** (instancias creadas por Main al consultar el colaborador 100 el 2024-01-15, que no conoce el proveedor actual y resuelve el nuevo fabricante).



Correspondencia UML – código

Elemento UML	Implementación
Interfaz AsistenciaTarget	public interface AsistenciaTarget
Realización Adapter -> Target	class AdaptadorAsistencia implements AsistenciaTarget
Asociación biblioteca (Adapter -> Adaptee)	private final BibliotecaBiometrica biblioteca, recibida en el constructor
Realización proveedor actual -> Target	class DispositivoProveedorActual implements AsistenciaTarget
Asociación dispositivos 0..* (Client -> Target)	private final List<AsistenciaTarget> dispositivos en SistemaAsistencia
Asociación servicio (Client -> Target)	private final AsistenciaTarget servicio en ConsolaUI
Dependencia getMarcaciones() -> Marcacion	List<Marcacion> getMarcaciones(long, Date, Date)
Optional - RegistroAsistencia	Optional<RegistroAsistencia>
Dato normalizado	RegistroAsistencia (inmutable, con LocalDate/LocalTime)

Código fuente

- **Repositorio:** https://github.com/NightMare2210/PDS_Actividad_02 (carpeta Escenario_2_Adapter/)
- **Lenguaje:** Java 17, Maven; pruebas con JUnit 5.

```
Escenario_2_Adapter/
├── pom.xml
└── src/
    ├── main/java/edu/universidad/asistencia/
    │   ├── target/AsistenciaTarget.java          # Target
    │   ├── adapter/AdaptadorAsistencia.java      # Adapter
    │   ├── adaptee/BibliotecaBiometrica.java     # Adaptee (biblioteca del fabricante, simulada)
    │   ├── adaptee/Marcacion.java                # Tipo de datos del fabricante
    │   ├── dispositivo/DispositivoProveedorActual.java  # Proveedor anterior (cumple el Target)
    │   ├── SistemaAsistencia.java                # Client (varios dispositivos)
    │   ├── domain/RegistroAsistencia.java        # Dato normalizado
    │   ├── ui/ConsolaUI.java                     # Client
    │   └── Main.java                             # Ensamblaje
    └── test/java/edu/universidad/asistencia/
        ├── adapter/AdaptadorAsistenciaTest.java  # 13 pruebas del Adapter
        ├── adapter/BibliotecaBiometricaFake.java # Doble de prueba del Adaptee
        ├── SistemaAsistenciaTest.java            # 4 pruebas de varios proveedores
        └── ui/ConsolaUITest.java                 # 5 pruebas del Client
```

```java
public interface AsistenciaTarget {
    Optional<RegistroAsistencia> obtenerRegistro(String idColaborador, LocalDate fecha);
}

public class BibliotecaBiometrica {          // Adaptee: no se modifica
    public List<Marcacion> getMarcaciones(long empleadoId, Date desde, Date hasta) { ... }
}

public class AdaptadorAsistencia implements AsistenciaTarget {

    private final BibliotecaBiometrica biblioteca;
    private final ZoneId zona;

    @Override
    public Optional<RegistroAsistencia> obtenerRegistro(String idColaborador, LocalDate fecha) {
        long empleadoId = Long.parseLong(idColaborador);                              // String -> long
        Date desde = Date.from(fecha.atStartOfDay(zona).toInstant());                 // 00:00:00.000
        Date hasta = Date.from(fecha.atTime(LocalTime.MAX).atZone(zona).toInstant()); // 23:59:59.999

        List<Marcacion> marcaciones = biblioteca.getMarcaciones(empleadoId, desde, hasta);

        LocalTime horaIngreso = null;
        LocalTime horaSalida = null;
        for (Marcacion m : marcaciones) {
            LocalTime hora = m.getTimestamp().toInstant().atZone(zona).toLocalTime();
            if ("ENTRADA".equalsIgnoreCase(m.getTipo())) {
                if (horaIngreso == null || hora.isBefore(horaIngreso)) horaIngreso = hora;
            } else if ("SALIDA".equalsIgnoreCase(m.getTipo())) {
                if (horaSalida == null || hora.isAfter(horaSalida)) horaSalida = hora;
            }
        }
        if (horaIngreso == null && horaSalida == null) return Optional.empty();
        return Optional.of(new RegistroAsistencia(idColaborador, fecha, horaIngreso, horaSalida));
    }
}
```

```java
public class SistemaAsistencia implements AsistenciaTarget {   // Client: solo conoce el Target
    private final List<AsistenciaTarget> dispositivos;

    @Override
    public Optional<RegistroAsistencia> obtenerRegistro(String idColaborador, LocalDate fecha) {
        for (AsistenciaTarget dispositivo : dispositivos) {
            Optional<RegistroAsistencia> registro = dispositivo.obtenerRegistro(idColaborador, fecha);
            if (registro.isPresent()) return registro;
        }
        return Optional.empty();
    }
}

// Main: Cliente -> Target <- Adapter -> Adaptee
AsistenciaTarget servicio = new SistemaAsistencia(List.of(
        proveedorActual,                                    // ya cumple el Target
        new AdaptadorAsistencia(new BibliotecaBiometrica())));    // nuevo fabricante, adaptado
new ConsolaUI(servicio, new Scanner(System.in), System.out).iniciar();
```

Ejecución:

```bash
cd Escenario_2_Adapter
mvn test                    # pruebas unitarias (22)
mvn compile exec:java       # aplicación de consola
```

Salida de la aplicación:

El colaborador 200 lo resuelve el proveedor anterior; el 100, el nuevo fabricante a través del adaptador, cuya biblioteca simulada responde con una entrada a las 08:00 y una salida a las 17:00.

```
==========================================
  Sistema de Consulta de Asistencia
==========================================
ID del colaborador (o 'salir' para terminar): 200
Fecha (yyyy-MM-dd) [Enter = hoy]: 2024-01-15

--- Registro de Asistencia ---
ID Colaborador : 200
Fecha          : 2024-01-15
Hora Ingreso   : 07:45:00
Hora Salida    : 16:30:00
------------------------------

ID del colaborador (o 'salir' para terminar): 100
Fecha (yyyy-MM-dd) [Enter = hoy]: 2024-01-15

--- Registro de Asistencia ---
ID Colaborador : 100
Fecha          : 2024-01-15
Hora Ingreso   : 08:00:00
Hora Salida    : 17:00:00
------------------------------

ID del colaborador (o 'salir' para terminar): salir
Hasta luego.
```

Uso crítico de inteligencia artificial

Herramienta utilizada: Claude Code (asistente de programación). La IA se usó como apoyo para diseñar, implementar y probar; las decisiones finales y la validación del comportamiento quedaron a cargo del grupo.

- **Prompt principal:**
  "Analiza el Escenario 02 del enunciado e identifica el problema estructural y el patrón que corresponde."

  “Implementa el patrón Adapter en Java 17 con Maven. Debe haber un contrato AsistenciaTarget con obtenerRegistro(String idColaborador, Date fecha) que devuelva un RegistroAsistencia con ID, fecha, hora de ingreso y hora de salida; una biblioteca del fabricante simulada (BibliotecaBiometrica, Marcacion) con getMarcaciones(long, Date, Date) que no debe modificarse; y un adaptador que convierta el ID a long, calcule el rango del día y reduzca la lista de marcaciones a ingreso y salida. Organízalo en los paquetes target, adapter, adaptee, domain y ui."

  "Agrega una interfaz de consola que use solo el contrato AsistenciaTarget, y pruebas unitarias con JUnit 5 que aíslen al adaptador mediante un doble de prueba de la biblioteca."

- **Recomendación aceptada:**

  Recomendación: Mantener el Adapter de objetos (composición) y concentrar en AdaptadorAsistencia toda la conversión entre los tipos del fabricante (long, java.util.Date, Marcacion con tipo en texto) y los del sistema institucional (String, LocalDate, LocalTime, RegistroAsistencia). Además, añadir DispositivoProveedorActual y SistemaAsistencia.

  Justificación: Con composición, el sistema solo ve obtenerRegistro(), el adaptador puede envolver cualquier instancia de la biblioteca (incluido un doble de prueba) y una nueva versión del SDK afecta a una sola clase; con herencia se expondrían los métodos del fabricante y se quedaría acoplado a su implementación. Concentrar la conversión de Date en el adaptador evita que java.util.Date se filtre al resto del sistema. Las dos clases nuevas demuestran un requisito del enunciado que el código original no evidenciaba: que el proveedor anterior y el nuevo convivan detrás del mismo contrato y que un tercero se incorpore sin modificar al cliente (SistemaAsistenciaTest.tercerProveedor).

- **Recomendación modificada o rechazada:**

  Recomendación de la IA: Al documentar el resultado, presentar el diagrama de clases con la distribución horizontal que Mermaid genera por defecto en la versión inicial (direction LR).

  Decisión modificada: El grupo pidió una distribución vertical. La primera versión vertical seguía siendo muy ancha, porque las tres implementaciones del Target quedaban en una sola fila, así que se reordenaron las relaciones de realización (AsistenciaTarget <|.. Implementación) para que el diagrama se organizara por capas: cliente, interfaz, implementaciones y, por último, la biblioteca y los datos. Se revisaron las imágenes antes de guardarlas; una línea sigue cruzando por detrás de una clase y se dejó así para no alterar el significado del diagrama.

  Justificación: la figura va en un documento de hoja vertical y una imagen muy ancha se vuelve ilegible al reducirla.

Material para uso académico exclusivo de Mauricio Ortiz Ochoa, mortizo@ups.edu.ec

Conclusiones de la actividad

- Ambos escenarios confirmaron que los patrones estructurales resuelven problemas de **organización y relación entre objetos**, no de algoritmos. En el Escenario 01, el presupuesto era una jerarquía parte-todo: con Composite, partidas y grupos comparten la abstracción ElementoPresupuestario, y el valor total y la estructura se obtienen por delegación recursiva, sin condicionales en el cliente y con niveles ilimitados.

- En el Escenario 02, el problema era una incompatibilidad de interfaces entre un sistema que no debe cambiar y una biblioteca que no puede cambiarse. Con un Adapter de objetos, toda la traducción de nombres, tipos y estructuras quedó en una sola clase, y el sistema institucional sigue dependiendo solo de su contrato.

- En los dos casos se cumplió el principio Abierto/Cerrado: se agregó una interfaz gráfica sin tocar las clases del Composite, y un nuevo fabricante se integra con otro adaptador sin modificar el cliente.

- Mantener la correspondencia entre UML y código exigió revisar ambos artefactos continuamente.

- La inteligencia artificial aceleró el diseño, la implementación y las pruebas, pero sus propuestas debieron validarse: en ambos escenarios el grupo rechazó una recomendación y corrigió diagramas generados, lo que demuestra que el criterio del equipo sigue siendo indispensable.

Material para uso académico exclusivo de Mauricio Ortiz Ochoa, mortizo@ups.edu.ec

Rúbrica de evaluación:

Criterios	No Presenta	Nivel Bajo	Nivel Medio	Nivel Alto
Selección, justificación y diagrama UML de Composite	0	1	2	3
Implementación funcional y correspondencia UML–código de Composite	0	1	2	3
Selección, justificación y diagrama UML de Adapter	0	1	2	3
Implementación funcional y correspondencia UML–código de Adapter	0	1	2	3
Uso crítico de IA, justificación y conclusiones	0	1	2	3

Documentos relacionados:

- Unidad_01_01_PDS.pdf
- Unidad_01_02_PDS.pdf
- Unidad_01_05_PDS.pdf
- Unidad_02_01_PDS.pdf

Material para uso académico exclusivo de Mauricio Ortiz Ochoa, mortizo@ups.edu.ec